

National University of Computer & Emerging Sciences

Computer Science Department

Course Code: CL-2006	Course: Operating System Lab
Instructor: Yasir Arfat	Semester: Spring 2025

Lab Manual – 08

Introduction to Multithreading & Synchronization with Pthreads

Table of Contents

Table of Contents	2
Objectives	4
Concepts	5
Processes vs Threads – What is the Difference?	5
How Threads Share Memory – The Memory Model	5
What is Pthreads?	6
Creating and Joining Threads	7
The Core Functions	7
Example 1: Your First Thread Program	7
What Happens If You Forget pthread_join()?	8
Why Do Threads Not Always Run in Order?	9
Example 2: Threads Running in Parallel (Infinite Loop)	9
Passing Arguments to Threads	11
Why Do We Need to Pass Arguments?	11
Example 3: Passing an Integer and Getting a Return Value	11
Example 4: Passing a Struct (Multiple Arguments)	12
Example 5: Passing an Array to a Thread	13
Thread Attributes	14
Example 6: Setting Thread Attributes	14
The Race Condition Problem	16
What Goes Wrong When Threads Share Data?	16
Example 7: Race Condition in Action (The Bug)	16
Solving Race Conditions with Mutex	18
What is a Mutex?	18
Example 8: Race Condition Fixed with Mutex	18
Common Mistakes & How to Fix Them	21
Mistake 1: Forgetting -lpthread	21
Mistake 2: Passing a Local Variable's Address to Multiple Threads	21
Mistake 3: Casting Errors with void*	21
Mistake 4: Not Protecting Shared Data	22
Pthreads Quick Reference	23
Compilation Reminder	23
Lab Tasks	24
Task 1: Thread Basics	24
Task 2: Parallel Array Sum	24

Task 3: Race Condition Demonstration	24
Task 4: Producer-Consumer with Mutex	24
Task 5: Thread vs Process Performance (Bonus)	25

Objectives

This lab introduces multithreading — one of the most important concepts in modern operating systems. Until now, you have created new processes with `fork()`. Now you will learn a lighter, faster alternative: threads. By the end of this lab, you will be able to:

- Understand the difference between a process and a thread
- Create threads using `pthread_create()` and wait for them with `pthread_join()`
- Pass arguments to threads (integers, pointers, structs, arrays)
- Get return values from threads
- Understand thread attributes (joinable vs detached, stack size, scheduling)
- Understand the race condition problem when threads share data
- Use mutex locks to protect shared resources and prevent race conditions
- Compile pthread programs with the `-lpthread` flag

Concepts

Processes vs Threads – What is the Difference?

In Lab 05, you learned about processes. When you call `fork()`, the OS creates a completely new process with its own separate memory space. This is like opening a brand new copy of a restaurant — it has its own kitchen, its own staff, its own inventory. Everything is duplicated.

A thread, on the other hand, is like hiring a new chef in the SAME kitchen. The new chef shares the same kitchen, same ingredients, same utensils — but works independently on their own dish. Threads exist inside a process and share everything: memory, open files, global variables. They only have their own stack (local variables) and program counter (where they are in the code).

Feature	Process (fork)	Thread (pthread)
Memory	Separate address space (copy of everything)	Shared address space (same memory)
Creation cost	Expensive (copy entire memory)	Cheap (just a new stack + registers)
Communication	Needs IPC (pipes, queues, shared memory)	Direct — threads share global variables
Crash isolation	One process crash does NOT affect others	One thread crash kills ALL threads in the process
Use case	Running separate programs	Parallelizing work within one program
System call	<code>fork()</code>	<code>pthread_create()</code>

OS CONCEPT

Threads are sometimes called “lightweight processes.” They are managed by the kernel scheduler just like processes, but creating a thread is 10–50x faster than creating a process because there is no memory copying. Modern programs (web servers, games, video players) use threads extensively to handle multiple tasks simultaneously within a single program.

How Threads Share Memory – The Memory Model

Understanding what threads share and what they do NOT share is the most important concept in this lab. Here is the complete picture:

Memory Area	Shared Between Threads?	What It Contains	Example
Global variables	YES — shared	Variables declared outside any function	<code>int counter = 0;</code> at the top of the file

Heap (malloc)	YES — shared	Dynamically allocated memory	<code>int *p = malloc(sizeof(int));</code>
Open files	YES — shared	File descriptors, sockets, pipes	<code>FILE *f = fopen("data.txt", "r");</code>
Code (text segment)	YES — shared	The compiled program instructions	All threads run functions from the same binary
Stack (local vars)	NO — separate	Local variables, function parameters	<code>int i</code> inside a thread function
Registers	NO — separate	CPU state (program counter, etc.)	Each thread has its own execution position
Thread ID	NO — separate	Unique identifier per thread	<code>pthread_self()</code> returns different values

WARNING

The shared global variables and heap are exactly WHY race conditions happen. If Thread A and Thread B both read and write the same global variable, they can overwrite each other's work. This is the central problem we solve with mutexes later in this lab.

What is Pthreads?

Pthreads (POSIX Threads) is the standard C library for creating and managing threads on Linux and Unix systems. It provides functions for creating threads, waiting for them, passing data, and synchronizing access to shared resources. To use it, you need:

- Include the header: `#include <pthread.h>`
- Link the library when compiling: `gcc program.c -o program -lpthread`

WARNING

If you forget the `-lpthread` flag when compiling, you will get “undefined reference to `pthread_create`” errors. This is the most common mistake students make. Always add `-lpthread` at the end of your gcc command.

Creating and Joining Threads

The Core Functions

You only need three functions to work with threads. Here is what each one does:

Function	What It Does	Analogy
<code>pthread_create(&tid, attr, func, arg)</code>	Create a new thread that runs <code>func(arg)</code>	Hiring a new chef and telling them what to cook
<code>pthread_join(tid, &retval)</code>	Wait for a thread to finish and get its result	Waiting for the chef to finish and collecting the dish
<code>pthread_exit(retval)</code>	End the current thread and return a value	The chef says “I’m done, here’s the dish”

Let us break down `pthread_create()` — the most important function:

```
int pthread_create(
    pthread_t *thread,          // OUTPUT: the thread ID (like a PID for threads)
    const pthread_attr_t *attr, // Thread attributes (NULL = defaults, which is fine)
    void *(*start_routine)(void *), // The FUNCTION the thread will run
    void *arg                   // The ARGUMENT to pass to that function
);
// Returns 0 on success, non-zero on failure
```

Example 1: Your First Thread Program

This is the simplest possible thread program. We create two threads, each runs a different function, and the main thread waits for both to finish:

Pthreads_1.c – Creating two simple threads

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

// This function will run in Thread 1
void *printp() {
    printf("printp\n"); // Thread 1 prints this
    return NULL;        // Thread is done, return nothing
}

// This function will run in Thread 2
void *print1() {
    printf("print1\n"); // Thread 2 prints this
```

```

    return NULL;
}

int main() {
    pthread_t t1, t2;        // Declare two thread IDs

    // Create Thread 1: runs printp() with no argument (NULL)
    pthread_create(&t1, NULL, printp, NULL);
    // Create Thread 2: runs print1() with no argument (NULL)
    pthread_create(&t2, NULL, print1, NULL);

    // Wait for both threads to finish before main() exits
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

```

$ gcc Pthreads_1.c -o pt1 -lpthread
$ ./pt1

```

Output

```

printp
print1

```

NOTE

Without `pthread_join()`, `main()` would exit immediately and kill both threads before they finish. Think of it like a restaurant manager who leaves before the chefs are done — the kitchen shuts down. `pthread_join()` tells `main` to WAIT.

What Happens If You Forget `pthread_join()`?

This is a common mistake. If you remove both `pthread_join()` calls from the program above, `main()` reaches `return 0` immediately and the entire process terminates — taking both threads down with it, even if they have not finished their work:

BAD – Threads killed before they finish

```

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, printp, NULL);
    pthread_create(&t2, NULL, print1, NULL);
    // NO pthread_join() calls!
    return 0; // main exits → process terminates → threads die!
}

```

```
}
```

Output

(nothing! – or maybe partial output if threads were fast enough)

The fix is simple: always call `pthread_join()` for every thread you create, unless you explicitly want a detached thread (covered later).

Why Do Threads Not Always Run in Order?

If you run Example 1 multiple times, sometimes you will see “printp” first, other times “print1” first. This is NOT a bug. The operating system’s scheduler decides which thread runs when. You created t1 before t2, but the scheduler might run t2 first because of CPU availability, context switching, or other factors.

OS CONCEPT

Thread execution order is non-deterministic. This means you cannot predict which thread will run first, second, or third. This is a fundamental property of concurrent programming. If your program depends on a specific order, you need synchronization mechanisms (mutexes, condition variables, barriers) — never assume threads will run in the order you created them.

Example 2: Threads Running in Parallel (Infinite Loop)

This example shows that threads truly run simultaneously. Two threads print different messages with `sleep()` delays, and you can see them interleaving:

Pthreads_2.c – Parallel execution with sleep

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void *printp1() {
    while (1) {
        printf("T1 = HELLO\n");
        sleep(2);           // Sleep 2 seconds
    }
}

void *printp2() {
    while (1) {
        sleep(2);           // Sleep 2 seconds first
        printf("T2 = WORLD\n");
    }
}
```

```
    }  
}  
  
int main() {  
    pthread_t t1, t2;  
    pthread_create(&t1, NULL, printp1, NULL);  
    pthread_create(&t2, NULL, printp2, NULL);  
    pthread_join(t1, NULL);    // This will never return (infinite loops)  
    pthread_join(t2, NULL);  
    return 0;  
}
```

Output

```
T1 = HELLO  
T2 = WORLD  
T1 = HELLO  
T2 = WORLD  
...    (press Ctrl+C to stop)
```

TIP

Both threads run at the SAME time. T1 prints, sleeps 2s, prints again. T2 sleeps 2s, prints, sleeps 2s, prints. They alternate because of the sleep timing. This is real parallel execution managed by the OS scheduler.

Passing Arguments to Threads

Why Do We Need to Pass Arguments?

In the examples above, our thread functions did not receive any data — they just printed a hardcoded string. In real programs, you need to tell each thread WHAT to work on. For example, you might create 4 threads to search different parts of an array, and each thread needs to know which part is theirs.

The last parameter of `pthread_create()` is `void *arg` — a generic pointer that lets you pass ANY type of data to the thread function. Let us see how:

Example 3: Passing an Integer and Getting a Return Value

This example creates two threads, passes a different integer to each, and collects their return values:

Pthreads_3.c – Passing integers, getting return values

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void *printp1(void *parm) {
    // Cast void* back to int* and dereference to get the value
    printf("HELLO = %d\n", *(int *)parm);
    return (void *)2;        // Return the integer 2 (cast to void*)
}

void *printp2(void *parm) {
    printf("Some Value = %d\n", *(int *)parm);
    return (void *)13;       // Return the integer 13
}

int main() {
    pthread_t t1, t2;
    int val1 = 1, val2 = 2;

    // Pass &val1 (address of val1) to thread 1
    pthread_create(&t1, NULL, printp1, (void *)&val1);
    // Pass &val2 (address of val2) to thread 2
    pthread_create(&t2, NULL, printp2, (void *)&val2);

    void *rc, *get;
    pthread_join(t1, &rc);    // Collect return value from thread 1
    pthread_join(t2, &get);   // Collect return value from thread 2
```

```

printf("Thread 1 returned: %d\n", (int)(long)rc);
printf("Thread 2 returned: %d\n", (int)(long)get);
return 0;
}

```

Output

```

HELLO = 1
Some Value = 2
Thread 1 returned: 2
Thread 2 returned: 13

```

Example 4: Passing a Struct (Multiple Arguments)

What if you need to pass multiple values of different types? Use a struct! This is the most common pattern in real-world thread programming:

Pthreads_4.c – Passing a struct to a thread

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Define a struct to hold multiple arguments
typedef struct {
    int int_arg;
    float float_arg;
    char str_arg[100];
} ThreadArgs;

void* loop(void* arg) {
    // Cast the void* back to our struct type
    ThreadArgs* args = (ThreadArgs*)arg;

    // Access all fields of the struct
    printf("int_arg = %d, float_arg = %.2f, str_arg = %s\n",
        args->int_arg, args->float_arg, args->str_arg);
    return NULL;
}

int main() {
    pthread_t t1;
    ThreadArgs args;
    args.int_arg = 42;
    args.float_arg = 3.14;
    strcpy(args.str_arg, "Hello, world!");
}

```

```

    // Pass the address of the struct
    pthread_create(&t1, NULL, loop, &args);
    pthread_join(t1, NULL);
    return 0;
}

```

Output

```
int_arg = 42, float_arg = 3.14, str_arg = Hello, world!
```

Example 5: Passing an Array to a Thread

Pthreads_5.c – Passing an array

```

#include <pthread.h>
#include <stdio.h>

void *array(void *arg) {
    int *arr = (int *)arg;    // Cast void* to int*
    for (int i = 0; i < 5; i++) {
        printf("%d ", *arr);  // Print each element
        arr++;                // Move to next element
    }
    printf("\n");
    return NULL;
}

int main() {
    int arr2[5] = {1, 2, 3, 4, 5};
    pthread_t t1;
    pthread_create(&t1, NULL, array, (void *)arr2);
    pthread_join(t1, NULL);
    return 0;
}

```

Output

```
1 2 3 4 5
```

Thread Attributes

So far, we have passed NULL as the second argument to `pthread_create()`. This uses default settings. But sometimes you want to control how a thread behaves:

Attribute	What It Controls	Default Value	Why Change It?
Detach state	Can you join this thread?	JOINABLE	Detached threads free resources automatically when done
Stack size	How much stack memory the thread gets	System default (usually 2–8 MB)	Increase for deeply recursive functions
Scheduling policy	How the OS schedules the thread	SCHED_OTHER (normal)	SCHED_FIFO or SCHED_RR for real-time
Scheduling priority	Priority relative to other threads	0	Higher = runs more often (needs root)

Example 6: Setting Thread Attributes

pthread_properties.c - Using thread attributes

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* myThreadFunc(void* arg) {
    for (int i = 0; i < 5; i++) {
        printf("Thread %d running\n", (int)(long)arg);
        usleep(100000); // Sleep 0.1 seconds
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;
    pthread_attr_t attr;           // Attribute object

    // Step 1: Initialize attributes with defaults
    pthread_attr_init(&attr);

    // Step 2: Set the attributes we want to change
    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);
    pthread_attr_setstacksize(&attr, 2 * 1024 * 1024); // 2 MB stack

    // Step 3: Create threads WITH our custom attributes
    pthread_create(&thread1, &attr, myThreadFunc, (void*)1);
```

```
pthread_create(&thread2, &attr, myThreadFunc, (void*)2);

// Step 4: Wait for threads to finish
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// Step 5: Clean up the attribute object
pthread_attr_destroy(&attr);
printf("Main thread finished.\n");
return 0;
}
```

TIP

In most cases, the default attributes (NULL) are perfectly fine. You only need custom attributes for specialized situations: real-time systems, deeply recursive algorithms, or fire-and-forget threads that you never want to join.

The Race Condition Problem

What Goes Wrong When Threads Share Data?

Here is where things get dangerous. Since all threads in a process share the same memory, what happens when two threads try to modify the same variable at the same time?

Real-world analogy: Imagine two cashiers at a store sharing one cash register. Cashier A reads the balance (\$100), adds \$50, and writes \$150. But at the EXACT same moment, Cashier B also reads \$100 (before A wrote \$150), adds \$30, and writes \$130. The final balance is \$130, but it should be \$180! The \$50 from Cashier A was lost. This is a race condition.

Example 7: Race Condition in Action (The Bug)

This program has 3 threads all incrementing the same global counter 5 times each. The expected result is 15 (3 threads × 5 increments). But run it multiple times and you might get different answers!

mutex.c – Race condition (BUGGY CODE)

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

int counter = 0;    // SHARED global variable – all threads read and write this!

void *printp1(void *parm) {
    int i;
    for (i = 0; i < 5; i++) {
        counter++;    // DANGER: multiple threads doing this at the same time!
        printf("thread %d counter %d\n", *(int*)parm, counter);
    }
    return NULL;
}

int main() {
    int a = 1, b = 2, c = 3;
    pthread_t t1, t2, t3;

    pthread_create(&t1, 0, printp1, (void*)&a);
    pthread_create(&t2, 0, printp1, (void*)&b);
    pthread_create(&t3, 0, printp1, (void*)&c);

    pthread_join(t1, 0);
    pthread_join(t2, 0);
}
```

```
pthread_join(t3, 0);

printf("Final counter: %d (expected 15)\n", counter);
return 0;
}
```

WARNING

Run this program multiple times: `gcc mutex.c -o mutex -lpthread && ./mutex`. You will see that the counter values are inconsistent — threads interleave and overwrite each other's increments. The final value might be 15 one time and 13 or 14 another time. This is a race condition.

Here is what the output might look like on two different runs — notice the inconsistency:

Run 1 (counter gets mixed up)	Run 2 (different mess)
thread 1 counter 1	thread 1 counter 1
thread 2 counter 2	thread 1 counter 2
thread 1 counter 3	thread 2 counter 2 ← DUPLICATE!
thread 3 counter 3 ← DUPLICATE!	thread 3 counter 4
thread 2 counter 5	thread 1 counter 5
...	...
Final counter: 14 ← WRONG!	Final counter: 13 ← WRONG!

The correct final value should be 15 (3 threads × 5 increments), but we get 14 or 13 because threads are overwriting each other's work. Every time you run it, you might get a different wrong answer — this unpredictability is the hallmark of a race condition.

OS CONCEPT

The operation `counter++` looks like one step, but at the CPU level it is THREE steps: (1) READ counter from memory into a register, (2) ADD 1 to the register, (3) WRITE the new value back to memory. If Thread A does step 1 and then Thread B does step 1 before A does step 3, both threads read the same old value — and one increment is lost. This is the fundamental reason race conditions exist.

Solving Race Conditions with Mutex

What is a Mutex?

A mutex (short for “mutual exclusion”) is a lock that ensures only ONE thread can access a critical section of code at a time. Think of it like a bathroom lock — when one person is inside and locks the door, everyone else has to wait outside until the lock is released.

Function	What It Does	Analogy
<code>pthread_mutex_init(&mutex, NULL)</code>	Create the lock	Install the lock on the bathroom door
<code>pthread_mutex_lock(&mutex)</code>	Lock it (or wait if already locked)	Person enters and locks the door
<code>pthread_mutex_unlock(&mutex)</code>	Unlock it (next waiting thread gets it)	Person exits and unlocks the door
<code>pthread_mutex_destroy(&mutex)</code>	Delete the lock (cleanup)	Remove the lock when the bathroom is demolished

Example 8: Race Condition Fixed with Mutex

Here is the same program, but now we use a mutex to protect the critical section (the counter++ operation). Only one thread can increment at a time:

mut_solved.c – Race condition FIXED with mutex

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

int counter = 0;           // Shared variable
pthread_mutex_t mutex;     // The lock!

void *printp1(void *parm) {
    int i;
    pthread_mutex_lock(&mutex);    // LOCK: "I'm going in, everyone wait!"

    for (i = 0; i < 5; i++) {
        counter++;                // Now SAFE – only one thread at a time
        printf("thread %d counter %d\n", *(int*)parm, counter);
        usleep(1000);             // Simulate some work
    }

    pthread_mutex_unlock(&mutex);  // UNLOCK: "I'm done, next person can go!"
}
```

```
    return NULL;
}

int main() {
    int a = 1, b = 2, c = 3;
    pthread_t t1, t2, t3;

    pthread_mutex_init(&mutex, NULL); // Create the lock

    pthread_create(&t1, 0, printp1, (void*)&a);
    pthread_create(&t2, 0, printp1, (void*)&b);
    pthread_create(&t3, 0, printp1, (void*)&c);

    pthread_join(t1, 0);
    pthread_join(t2, 0);
    pthread_join(t3, 0);

    pthread_mutex_destroy(&mutex); // Clean up the lock
    printf("Final counter: %d\n", counter); // Always 15!
    return 0;
}
```

Output

```
thread 1 counter 1
thread 1 counter 2
thread 1 counter 3
thread 1 counter 4
thread 1 counter 5
thread 2 counter 6
thread 2 counter 7
thread 2 counter 8
thread 2 counter 9
thread 2 counter 10
thread 3 counter 11
thread 3 counter 12
thread 3 counter 13
thread 3 counter 14
thread 3 counter 15
Final counter: 15
```

Now the counter is always 15, no matter how many times you run it! Each thread waits for the lock before entering the critical section.

NOTE

In the solved version, the threads run one after another (not truly parallel) because the entire loop is locked. In real programs, you want to lock **ONLY** the minimum code necessary (just the `counter++` line), so threads can still run in parallel for the rest of their work. This is called fine-grained locking.

Common Mistakes & How to Fix Them

Almost every student hits these errors when first working with pthreads. Here is what they look like and how to fix them:

Mistake 1: Forgetting -lpthread

Error you see

```
$ gcc program.c -o program
/usr/bin/ld: /tmp/ccXYZ.o: in function `main':
program.c: undefined reference to `pthread_create'
collect2: error: ld returned 1 exit status
```

Fix: Add -lpthread at the end of your gcc command: gcc program.c -o program -lpthread

Mistake 2: Passing a Local Variable's Address to Multiple Threads

Buggy code – all threads see the SAME value

```
for (int i = 0; i < 5; i++) {
    pthread_create(&threads[i], NULL, func, &i); // BUG!
    // Problem: all 5 threads get a pointer to the SAME variable 'i'
    // By the time a thread reads *arg, 'i' might already be 5!
}
```

Fix – use a separate variable for each thread

```
int args[5];
for (int i = 0; i < 5; i++) {
    args[i] = i; // Each thread gets its OWN copy
    pthread_create(&threads[i], NULL, func, &args[i]); // CORRECT
}
```

Mistake 3: Casting Errors with void*

Thread functions take and return void*. Students often get confused about casting. Here is the pattern:

```
// Passing an int TO a thread:
int val = 42;
pthread_create(&t, NULL, func, (void *)&val); // Pass ADDRESS

// Inside the thread function, get the int BACK:
void *myFunc(void *arg) {
```

```
int received = *(int *)arg; // Cast void* to int*, then dereference
}
```

Mistake 4: Not Protecting Shared Data

If multiple threads read AND write the same variable and you do not use a mutex, you have a race condition. The program might work 99 times and fail on the 100th run. Always use `pthread_mutex_lock/unlock` around shared variable access.

TIP

A good rule of thumb: if any global variable is modified by more than one thread, protect it with a mutex. Read-only shared data (constants) does not need a mutex.

Pthreads Quick Reference

Function	Header	Purpose
pthread_create(&tid, attr, func, arg)	<pthread.h>	Create a new thread running func(arg)
pthread_join(tid, &retval)	<pthread.h>	Wait for thread to finish, collect return value
pthread_exit(retval)	<pthread.h>	Terminate the current thread, return a value
pthread_self()	<pthread.h>	Get the thread ID of the calling thread
pthread_equal(t1, t2)	<pthread.h>	Compare two thread IDs (never use ==)
pthread_attr_init(&attr)	<pthread.h>	Initialize a thread attributes object
pthread_attr_setdetachstate()	<pthread.h>	Set joinable (default) or detached
pthread_attr_setstacksize()	<pthread.h>	Set the thread's stack size
pthread_attr_destroy(&attr)	<pthread.h>	Destroy the attributes object (cleanup)
pthread_mutex_init(&m, NULL)	<pthread.h>	Create a mutex lock
pthread_mutex_lock(&m)	<pthread.h>	Lock the mutex (blocks if already locked)
pthread_mutex_unlock(&m)	<pthread.h>	Unlock the mutex
pthread_mutex_destroy(&m)	<pthread.h>	Destroy the mutex (cleanup)

Compilation Reminder

```
# ALWAYS add -lpthread when compiling pthread programs!
$ gcc program.c -o program -lpthread

# With warnings enabled (recommended):
$ gcc -Wall program.c -o program -lpthread
```

Lab Tasks

Complete the following tasks. Submit your source code and screenshots of compilation and output.

Task 1: Thread Basics

1. Write a program that creates 3 threads. Each thread prints “I am thread X” where X is 1, 2, or 3 (pass the number as an argument). The main thread waits for all three and then prints “All threads done.”
2. Run the program 5 times. Do the threads always print in the same order? Explain why or why not.

Task 2: Parallel Array Sum

1. Create an array of 1,000,000 random integers.
2. Create 4 threads. Each thread sums a different quarter of the array (thread 1 sums indices 0–249999, thread 2 sums 250000–499999, etc.).
3. Use a struct to pass each thread its start index, end index, and a pointer to the array.
4. Collect the partial sums from each thread and print the total. Verify it matches a single-threaded sum.

Task 3: Race Condition Demonstration

1. Write a program where 5 threads each increment a shared global counter 100,000 times (no mutex).
2. Run it 10 times and record the final counter value each time. Is it always 500,000? Why not?
3. Now add a mutex around the counter++ operation and run again. Is it always 500,000 now? Explain.

Task 4: Producer-Consumer with Mutex

1. Create a shared buffer (an integer variable). One “producer” thread writes values 1 through 10 into the buffer (with a small sleep between each).
2. A “consumer” thread reads and prints the buffer value repeatedly.
3. First run WITHOUT mutex. Observe: does the consumer sometimes read the same value twice or miss values?
4. Then add mutex protection. Does it improve? Discuss the limitations (hint: you need condition variables for a proper solution — this is a preview of a future topic).

Task 5: Thread vs Process Performance (Bonus)

1. Write two programs that both create 100 children: one using `fork()` (100 processes) and one using `pthread_create()` (100 threads).
2. Each child/thread should do some work (e.g., compute a sum).
3. Use the `time` command to measure: `time ./fork_version` vs `time ./thread_version`.
4. Compare the “real” time. Which is faster? By how much? Explain why.

— End of Lab 09 —